Assignment 5

Nirupam Das
2024CSB108
2nd Year, 4th Semester

March 2, 2026

Contents

1 Problem Statement 1 **2**

2 Problem Statement 2 **8**

3 Problem Statement 3 **15**

1 Problem Statement 1

Let $S = \{0, 1, 2, \ldots, n-1\}$ be a set of n distinct elements. Initially, each element forms its own separate set, so the collection consists of n disjoint singleton sets. This initial configuration is represented using a parent array P of size n, where $P[i] = i$ for all $0 \leq i \leq n-1$, and a rank array R, where $R[i] = 0$ for all $0 \leq i \leq n-1$.

Design and implement an efficient **Disjoint Set (Union–Find)** data structure that supports two fundamental operations: $Find(x)$, which returns the representative (root) of the set containing element x, and $Union(x, y)$, which merges the two sets containing elements x and y. To improve efficiency, apply **path compression** during the $Find$ operation so that each visited node directly points to the root, and apply **union by rank** during the $Union$ operation so that the tree with smaller height is attached under the tree with larger height, thereby keeping the tree shallow.

Code

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct DSU {
    int *parent;
    int *rank;
} DSU;

void fillZero(DSU *d, int size) {
    for (int i = 0; i < size; i++) {
        d->rank[i] = 0;
    }
}

void fillParent(DSU *d, int size) {
    for (int i = 0; i < size; i++) {
        d->parent[i] = i;
    }
}

void init(DSU *d, int size) {
    d->parent = (int *)malloc(sizeof(int) * size);
    fillParent(d, size);
    d->rank = (int *)malloc(sizeof(int) * size);
    fillZero(d, size);
}

void freeDSU(DSU *d) {
    if (d->parent) free(d->parent);
    if (d->rank) free(d->rank);
    d->parent = NULL;
    d->rank = NULL;
}

int find(DSU *d, int x) {
    if (d->parent[x] == x)
        return x;
    return d->parent[x] = find(d, d->parent[x]);
}
```

```c

int Union(DSU *d, int x, int y) {
    int root1 = find(d, x);
    int root2 = find(d, y);

    if (root1 == root2)
        return root1;
    else if (d->rank[root1] == d->rank[root2]) {
        d->parent[root2] = root1;
        (d->rank[root1])++;
        return root1;
    } else {
        if (d->rank[root1] > d->rank[root2]) {
            d->parent[root2] = root1;
            return root1;
        } else {
            d->parent[root1] = root2;
            return root2;
        }
    }
}

void display(DSU *d, int size) {
    printf("\nIndex:  ");
    for (int i = 0; i < size; i++)
        printf("%d ", i);
    printf("\nParent: ");
    for (int i = 0; i < size; i++)
        printf("%d ", d->parent[i]);
    printf("\nRank:   ");
    for (int i = 0; i < size; i++)
        printf("%d ", d->rank[i]);
    printf("\n");
}

int main() {
    DSU myDsu;
    myDsu.parent = NULL;
    myDsu.rank = NULL;

    int size, choice, x, y;

    printf("Enter the number of elements (n): ");
    if (scanf("%d", &size) != 1)
        return 1;

    init(&myDsu, size);
    printf("DSU initialized with %d elements (0 to %d).\n", size, size -
    1);

    while (1) {
        printf("\nChoose an option:");
        printf("\n1. Union(x, y)");
        printf("\n2. Find(x)");
        printf("\n3. Check if x and y are in same set");
        printf("\n4. Display Arrays");
        printf("\n5. Exit");
        printf("\nChoice: ");
```

```c
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter two elements to union: ");
                scanf("%d %d", &x, &y);
                if (x >= 0 && x < size && y >= 0 && y < size) {
                    int root = Union(&myDsu, x, y);
                    printf("Union successful. New root is: %d\n", root);
                } else {
                    printf("Error: Elements out of bounds.\n");
                }
                break;

            case 2:
                printf("Enter element to find: ");
                scanf("%d", &x);
                if (x >= 0 && x < size) {
                    printf("Root of %d is: %d\n", x, find(&myDsu, x));
                } else {
                    printf("Error: Element out of bounds.\n");
                }
                break;

            case 3:
                printf("Enter two elements to check: ");
                scanf("%d %d", &x, &y);
                if (x >= 0 && x < size && y >= 0 && y < size) {
                    if (find(&myDsu, x) == find(&myDsu, y))
                        printf("Common Head = (%d).\n", find(&myDsu, x));
                    else
                        printf("Head of %d: %d\nHead of %d: %d\n", x, find(&myDsu, x), y, find(&myDsu, y));
                }
                break;

            case 4:
                display(&myDsu, size);
                break;

            case 5:
                freeDSU(&myDsu);
                printf("Memory freed. Exiting...\n");
                return 0;

            default:
                printf("Invalid choice.\n");
        }
    }
    return 0;
}
```

Output

```
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$ ./a5Q1
Enter the number of elements (n): 7
DSU initialized with 7 elements (0 to 6).

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 1
Enter two elements to union: 0 3
Union successful. New root is: 0

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 1
Enter two elements to union: 3 4
Union successful. New root is: 0

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 1
Enter two elements to union: 4 6
Union successful. New root is: 0

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 1
Enter two elements to union: 2 5
Union successful. New root is: 2
```

```
Choice: 1
Enter two elements to union: 2 5
Union successful. New root is: 2

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 1
Enter two elements to union: 1 5
Union successful. New root is: 2

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 4

Index:  0 1 2 3 4 5 6
Parent: 0 2 2 0 0 2 0
Rank:   1 0 1 0 0 0 0

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 2
Enter element to find: 3 2
Root of 3 is: 0

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: Enter element to find: 3
Root of 3 is: 0
```

```
Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 3
Enter two elements to check: 3 0
Common Head =(0).

Choose an option:
1. Union(x, y)
2. Find(x)
3. Check if x and y are in same set
4. Display Arrays
5. Exit
Choice: 5
Memory freed. Exiting...
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$
```

Figure 1: Output results for Problem Statement 1

2 Problem Statement 2

In a large school campus, thousands of students dynamically form different friend groups, where new friendships are created daily and teachers frequently need to verify whether two students belong to the same group. Initially, each student forms an independent group. As interactions increase, multiple groups gradually merge, and it becomes essential to efficiently manage these dynamic group connections.

Given n students and m randomly generated friendship operations, design and implement an optimized **Disjoint Set (Union–Find)** data structure using **dynamic arrays**, which supports two fundamental operations:

- **Union**(x, y): Merges the sets containing students x and y.
- **Find**(x): Returns the representative (leader) of the set containing student x.

Let the initial parent array be defined as:

$$P[i] = i, \quad \text{for all } 0 \leq i \leq n-1$$

To enhance performance, apply **path compression** during the Find operation and **union by rank** during the Union operation. The system must efficiently process all m operations, ensuring fast connectivity queries even for large values of n and m.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct DSU {
    int *parent;
    int *rank;
} DSU;

void fillZero(DSU *d, int size) {
    for (int i = 0; i < size; i++) {
        d->rank[i] = 0;
    }
}

void fillParent(DSU *d, int size) {
    for (int i = 0; i < size; i++) {
        d->parent[i] = i;
    }
}

void init(DSU *d, int size) {
    d->parent = (int *)malloc(sizeof(int) * size);
    fillParent(d, size);
    d->rank = (int *)malloc(sizeof(int) * size);
    fillZero(d, size);
}

void freeDSU(DSU *d) {
```

```c
    if (d->parent) free(d->parent);
    if (d->rank) free(d->rank);
    d->parent = NULL;
    d->rank = NULL;
}

int find(DSU *d, int x) {
    if (d->parent[x] == x)
        return x;
    return d->parent[x] = find(d, d->parent[x]);
}

int Union(DSU *d, int x, int y) {
    int root1 = find(d, x);
    int root2 = find(d, y);

    if (root1 == root2)
        return root1;
    else if (d->rank[root1] == d->rank[root2]) {
        d->parent[root2] = root1;
        (d->rank[root1])++;
        return root1;
    } else {
        if (d->rank[root1] > d->rank[root2]) {
            d->parent[root2] = root1;
            return root1;
        } else {
            d->parent[root1] = root2;
            return root2;
        }
    }
}

void display(DSU *d, int size) {
    printf("\nIdx | Parent | Rank\n");
    printf("-------------------\n");

    int limit = (size > 15) ? 10 : size;

    for (int i = 0; i < limit; i++) {
        printf("%3d | %6d | %4d\n", i, d->parent[i], d->rank[i]);
    }

    if (size > 15) {
        printf("... | ...... | ....\n");
        for (int i = size - 5; i < size; i++) {
            printf("%3d | %6d | %4d\n", i, d->parent[i], d->rank[i]);
        }
    }
    printf("\n");
}

int main() {
    srand(time(NULL));
    DSU student;
    student.parent = NULL;
    student.rank = NULL;

```

```c
    int size, operation_count, x, y, choice;
    printf("Enter the number of elements (n): ");
    if (scanf("%d", &size) != 1)
        return 1;

    init(&student, size);
    printf("DSU initialized with %d elements (0 to %d).\n", size, size - 1);
    display(&student, size);

    printf("Enter the number of Union operations: ");
    scanf("%d", &operation_count);

    while (operation_count--) {
        printf("\n====================================\n");
        x = rand() % size;
        y = rand() % size;

        printf("Action: Union(%d, %d)\n", x, y);
        Union(&student, x, y);
        display(&student, size);
        printf("====================================\n");
    }

    do {
        printf("\nFind representative of any student of any group after the "
                "operations [-1 to quit]: ");
        scanf("%d", &choice);
        if (choice >= 0 && choice < size)
            printf("Representative of %d: %d\n", choice, find(&student, choice));
        else if (choice != -1)
            printf("Invalid index.\n");
    } while (choice != -1);

    freeDSU(&student);
    return 0;
}
```

Output

```
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$ ./a5q2
Enter the number of elements (n): 10
DSU initialized with 10 elements (0 to 9).

Idx | Parent | Rank
------------------
  0 |      0 |    0
  1 |      1 |    0
  2 |      2 |    0
  3 |      3 |    0
  4 |      4 |    0
  5 |      5 |    0
  6 |      6 |    0
  7 |      7 |    0
  8 |      8 |    0
  9 |      9 |    0

Enter the number of Union operations: 5

====================================
Action: Union(4, 6)

Idx | Parent | Rank
------------------
  0 |      0 |    0
  1 |      1 |    0
  2 |      2 |    0
  3 |      3 |    0
  4 |      4 |    1
  5 |      5 |    0
  6 |      4 |    0
  7 |      7 |    0
  8 |      8 |    0
  9 |      9 |    0

====================================

====================================
Action: Union(8, 5)

Idx | Parent | Rank
------------------
  0 |      0 |    0
  1 |      1 |    0
  2 |      2 |    0
```

```
====================================

====================================
Action: Union(8, 5)

Idx | Parent | Rank
-------------------
  0 |      0 |    0
  1 |      1 |    0
  2 |      2 |    0
  3 |      3 |    0
  4 |      4 |    1
  5 |      8 |    0
  6 |      4 |    0
  7 |      7 |    0
  8 |      8 |    1
  9 |      9 |    0

====================================

====================================
Action: Union(2, 7)

Idx | Parent | Rank
-------------------
  0 |      0 |    0
  1 |      1 |    0
  2 |      2 |    1
  3 |      3 |    0
  4 |      4 |    1
  5 |      8 |    0
  6 |      4 |    0
  7 |      2 |    0
  8 |      8 |    1
  9 |      9 |    0

====================================

====================================
Action: Union(1, 0)

Idx | Parent | Rank
-------------------
  0 |      1 |    0
  1 |      1 |    1
```

```
=====================================

=====================================
Action: Union(1, 0)

Idx | Parent | Rank
-------------------
  0 |      1 |    0
  1 |      1 |    1
  2 |      2 |    1
  3 |      3 |    0
  4 |      4 |    1
  5 |      8 |    0
  6 |      4 |    0
  7 |      2 |    0
  8 |      8 |    1
  9 |      9 |    0

=====================================

=====================================
Action: Union(3, 5)

Idx | Parent | Rank
-------------------
  0 |      1 |    0
  1 |      1 |    1
  2 |      2 |    1
  3 |      8 |    0
  4 |      4 |    1
  5 |      8 |    0
  6 |      4 |    0
  7 |      2 |    0
  8 |      8 |    1
  9 |      9 |    0

=====================================

Find representative of any student of any group after the operations [-1 to quit]: 2
Representative of 2: 2

Find representative of any student of any group after the operations [-1 to quit]: 3
Representative of 3: 8
```

```
Find representative of any student of any group after the operations [-1 to quit]: 2
Representative of 2: 2

Find representative of any student of any group after the operations [-1 to quit]: 3
Representative of 3: 8

Find representative of any student of any group after the operations [-1 to quit]: 4
Representative of 4: 4

Find representative of any student of any group after the operations [-1 to quit]: 5
Representative of 5: 8

Find representative of any student of any group after the operations [-1 to quit]: 6
Representative of 6: 4

Find representative of any student of any group after the operations [-1 to quit]: 7
Representative of 7: 2

Find representative of any student of any group after the operations [-1 to quit]: 8
Representative of 8: 8

Find representative of any student of any group after the operations [-1 to quit]: 9
Representative of 9: 9

Find representative of any student of any group after the operations [-1 to quit]: 10
Invalid index.

Find representative of any student of any group after the operations [-1 to quit]: -1
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$
```

Figure 2: Output results for Problem Statement 2

3 Problem Statement 3

A city contains N buildings, initially all isolated, which can be mathematically modeled as N disjoint singleton sets, where the parent array is initialized as

$$P[i] = i, \quad \text{for all } 0 \leq i \leq N-1.$$

Over time, M road construction operations are performed, where each operation connects two buildings (u, v), with $0 \leq u, v \leq N-1$. To efficiently manage these dynamic connections, design a **Disjoint Set (Union–Find)** data structure supporting two operations: $Union(u, v)$, which merges the sets containing u and v, and $Find(x)$, which returns the representative (leader) of the set containing x. After each union operation, the system must verify whether the entire city has become fully connected, which mathematically implies that all buildings belong to a single connected component, i.e., there exists a unique representative r such that

$$Find(i) = r, \quad \text{for all } 0 \leq i \leq N-1.$$

Code

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct DSU {
    int *parent;
    int *rank;
    int components; // To track the number of disconnected sets at any point
} DSU;

void init(DSU *d, int size) {
    d->parent = (int *)malloc(sizeof(int) * size);
    d->rank = (int *)malloc(sizeof(int) * size);
    d->components = size;

    for (int i = 0; i < size; i++) {
        d->parent[i] = i;
        d->rank[i] = 0;
    }
}

void freeDSU(DSU *d) {
    if (d->parent) free(d->parent);
    if (d->rank) free(d->rank);
    d->parent = NULL;
    d->rank = NULL;
}

int find(DSU *d, int x) {
    if (d->parent[x] == x)
        return x;

    return d->parent[x] = find(d, d->parent[x]);
}

```

```c
int Union(DSU *d, int x, int y) {
    int root1 = find(d, x);
    int root2 = find(d, y);

    if (root1 == root2) {
        return root1;
    }

    d->components--;

    if (d->rank[root1] < d->rank[root2]) {
        d->parent[root1] = root2;
        return root2;
    } else if (d->rank[root1] > d->rank[root2]) {
        d->parent[root2] = root1;
        return root1;
    } else {
        d->parent[root2] = root1;
        d->rank[root1]++;
        return root1;
    }
}

int main() {
    int n;
    printf("Enter the total number of buildings initially (N): ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input for N.\n");
        return 1;
    }

    DSU city;
    init(&city, n);

    int choice, u, v, leader;

    while (1) {
        printf("\nChoose an option:\n");
        printf("1. Construct Road (Union)\n");
        printf("2. Find Building Representative (Find)\n");
        printf("3. Exit\n");
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1) break;

        switch (choice) {
            case 1:
                printf("Enter buildings to connect (u v, from 0 to %d): ", n - 1);
                scanf("%d %d", &u, &v);

                if (u < 0 || u >= n || v < 0 || v >= n) {
                    printf("Invalid building numbers.\n");
                    break;
                }

                if (find(&city, u) == find(&city, v)) {
                    printf("Buildings %d and %d are already connected.\n
```

```
", u, v);
                } else {
                    Union(&city, u, v);
                    printf("Road constructed successfully between %d and %d.\n", u, v);
                }

                if (city.components == 1) {
                    printf("The entire city is now Fully Connected.\n");
                } else {
                    printf("City is not fully connected yet. Remaining components: %d\n", city.components);
                }
                break;

            case 2:
                printf("Enter building number to find its leader: ");
                scanf("%d", &u);
                if (u < 0 || u >= n) {
                    printf("Invalid building number.\n");
                } else {
                    leader = find(&city, u);
                    printf("The representative (leader) for building %d is %d.\n", u, leader);
                }
                break;

            case 3:
                freeDSU(&city);
                printf("Memory freed. Exiting program...\n");
                return 0;

            default:
                printf("Please select 1, 2, or 3.\n");
        }
    }

    freeDSU(&city);
    return 0;
}
```

Output

```
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$ ./a5q3
Enter the total number of buildings initially (N): 5

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 1
Enter buildings to connect (u v, from 0 to 4): 0 4
Road constructed successfully between 0 and 4.
City is not fully connected yet. Remaining components: 4

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 1
Enter buildings to connect (u v, from 0 to 4): 2 3
Road constructed successfully between 2 and 3.
City is not fully connected yet. Remaining components: 3

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 1
Enter buildings to connect (u v, from 0 to 4): 1 4
Road constructed successfully between 1 and 4.
City is not fully connected yet. Remaining components: 2

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 1
Enter buildings to connect (u v, from 0 to 4): 3 1
Road constructed successfully between 3 and 1.
The entire city is now Fully Connected.

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 2
Enter building number to find its leader: 3
```

```
Enter building number to find its leader: 3
The representative (leader) for building 3 is 2.

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 2
Enter building number to find its leader: 1
The representative (leader) for building 1 is 2.

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 1
Enter buildings to connect (u v, from 0 to 4): 0 4
Buildings 0 and 4 are already connected.
The entire city is now Fully Connected.

Choose an option:
1. Construct Road (Union)
2. Find Building Representative (Find)
3. Exit
Enter choice: 3
Memory freed. Exiting program...
(base) nirupam-das@ND:~/Documents/4-th sem labs/Algo Lab/Assignment5$
```

Figure 3: Output results for Problem Statement 3